

Data-driven Physical Modelling Coursework – Part 1

Note:

- Make sure the data file is in the same directory as the .ipynb file
- Below are the following packages used in the entirety of Coursework Part 2 including the seeds
- Written times in results for codes that took long to run

```
In [ ]: import numpy as np
import random
import numpy.linalg as la
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from itertools import combinations_with_replacement
import math
```

Problem 1

Question 1

```
In [ ]: def split_data(data_filepath: str, test_fraction: float = 0.2, seed: int = 65,
                      output_filename: str = 'split_data_random.npz'):

    np.random.seed(seed)

    data_dict = np.load(data_filepath)
    sorted_traj = sorted(data_dict.keys())

    # Load each trajectory
    data_all = [data_dict[traj] for traj in sorted_traj]

    num_traj = len(data_all)
    num_test = int(test_fraction * num_traj)

    indices = np.arange(num_traj)
    np.random.shuffle(indices)

    test_idx = indices[:num_test]
    train_idx = indices[num_test:]

    train_traj = [data_all[i] for i in train_idx]
    test_traj = [data_all[i] for i in test_idx]

    np.savez(output_filename,
              train_trajectories=train_traj,
              test_trajectories=test_traj)

    return train_traj, test_traj
```

Question 2

The first method is a trajectory split, which involves randomly selecting 20% of the total number of trajectories as the testing data and the rest as training. This method allows each trajectory to stay complete so that the model is evaluated on a new set of unseen data however the drawback is that if the number of trajectories decreases, there are much fewer datapoints remaining for training and testing which could significantly reduce accuracy or increase the effects of overfitting to the testing data.

The second method is a temporal split, where X and Y delay embedded matrices are formed and 20% of the same columns, representing the individual timesteps, are selected from both matrices as the testing data and the rest as training data. This method is valid for predictions one time step ahead. The problem is that the testing points can be very close to the training point, causing data leakage hence may not be reliable. The test set would not be independent of the training set, so performance metrics would be unrealistic.

Question 3

a) The accuracy must be tested in the physical coordinate system. PCA follows an autoencoder model where it uses an encoder, given by the matrix $\mathbf{U}$ which maps the physical state,

$$x \in X$$

into the reduced coordinates,

$$z = \mathbf{U}^T x \in Z.$$

In this reduced space, Z , the best reduced order model is constructed and is decoded back to X , where the error can be measured. It cannot be measured in the reduced space since there are only latent coordinates in Z , so they have no physical meaning. Hence, to assess the accuracy, it must be tested in the physical space.

b) RMSE can be used as shown below:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{k=1}^N (\hat{x}_k - x_k)^2},$$

where

$$\frac{1}{N}$$

normalises the total error over the number of data points. This is necessary since different delay lengths computes different number of data points, and the normalisation allows the RMSE to be comparable over varying PCA ranks and delay lengths since it is the average RMSE per datapoint.

Question 4

```

In [ ]: train_trajectories, test_trajectories = split_data( # Loading all trajectories and
    data_filepath="double_beam_data.npz",
    test_fraction=0.2,
    seed=18,
    output_filename="split_data_random.npz"
)

def delay_embedding(traj_list, delay): # Delay embedding for each trajectory
    X_list = []
    Y_list = []

    for traj in traj_list:
        traj = np.ascontiguousarray(traj) # Just ensures that the array is stored
        T = traj.shape[1]
        # Number of windows must be the same for X and Y
        window_count = T - delay

        if window_count <= 0:
            continue
        X = np.vstack([traj[:, i : i + window_count] for i in range(delay)])
        Y = np.vstack([traj[:, i + 1 : i + 1 + window_count] for i in range(delay)])

        X_list.append(X)
        Y_list.append(Y)

    if len(X_list) == 0:
        return np.zeros((delay, 0)), np.zeros((delay, 0))

    return np.hstack(X_list), np.hstack(Y_list) # Concatenate all trajectories

# Setting grid to find best combination of delay length and PCA rank
max_delay = 32
max_rank = 16

all_delays = range(1, max_delay + 1)
all_ranks = range(1, max_rank + 1)

pca_error_train = np.zeros((len(all_delays), len(all_ranks)))
pca_error_test = np.zeros((len(all_delays), len(all_ranks)))

# Loops through all delays
for di, delay in enumerate(all_delays):

    X_train, Y_train = delay_embedding(train_trajectories, delay)
    X_test, Y_test = delay_embedding(test_trajectories, delay)

    # Skip invalid embeddings
    if X_train.size == 0 or Y_train.size == 0 or X_test.size == 0 or Y_test.size == 0:
        pca_error_train[di, :] = np.nan
        pca_error_test[di, :] = np.nan
        continue

```

```

C = X_train @ X_train.T          # Covariance matrix
U, S, Vt = la.svd(C, hermitian=True) # Carries out SVD on covariance matrix

# Loops through all ranks
for ri, rank in enumerate(all_ranks):

    # Skip invalid combinations
    if rank > delay:
        pca_error_train[di, ri] = np.nan
        pca_error_test[di, ri] = np.nan
        continue

    Ur = U[:, :rank] # Extracts first number of principle components

    # Project training data into reduced PCA space
    Xtr_r = Ur.T @ X_train
    Ytr_r = Ur.T @ Y_train

    W_r = (Ytr_r @ Xtr_r.T) @ la.pinv(Xtr_r @ Xtr_r.T, rcond=1e-9) # Least squares
    W_phys = Ur @ W_r @ Ur.T # Reduced model

    # Approximated predictions using Learned Linear operator
    Ytr_pred = W_phys @ X_train
    Yte_pred = W_phys @ X_test

    # Calculating distances between predicted and true values
    dis_tr = Ytr_pred[-1, :] - Y_train[-1, :]
    dis_te = Yte_pred[-1, :] - Y_test[-1, :]

    # RMSE to find best combination of delay and rank
    pca_error_train[di, ri] = np.sqrt(np.mean(dis_tr**2))
    pca_error_test[di, ri] = np.sqrt(np.mean(dis_te**2))

valid_combinations = ~np.isnan(pca_error_test) # Filter out any invalid combinations

if not np.any(valid_combinations):
    print("No valid delay/rank combination produced usable embeddings.")
else:
    best_flat_index = np.argmin(pca_error_test[valid_combinations]) # Minimum testing error
    valid_indices = np.argwhere(valid_combinations) # Finds best combination

    best_d, best_r = valid_indices[best_flat_index] # Index of best delay and rank

    best_pca_delay = all_delays[best_d] # Actual delay length
    best_pca_rank = all_ranks[best_r] # Actual PCA rank
    best_error = pca_error_test[best_d, best_r] # Best testing error

    print(f"The best model uses a delay length of {best_pca_delay} and a PCA rank of {best_pca_rank} which gives the lowest RMSE of {best_error}")

```

Result: The best model uses a delay length of 9 and a PCA rank of 6 which gives the lowest RMSE of 8.99e-06

Question 5

```

In [ ]: # Convert ranges to arrays for plotting
delays = np.array(list(all_delays))
ranks  = np.array(list(all_ranks))

# Used log graphs for better visualisation due to wide order differences in RMSE va
log_train = np.log10(np.ma.masked_invalid(pca_error_train))
log_test  = np.log10(np.ma.masked_invalid(pca_error_test))

# Obtaining limits for shared colour scale using minimum and maximum from both erro
min_scale = np.nanmin([log_train, log_test])
max_scale = np.nanmax([log_train, log_test])

plt.figure(figsize=(14, 6))
plt.suptitle("Evaluating RMSE Against Varying PCA Ranks and Delay Lengths", fontsize=14)

# Training error heatmap
plt.subplot(1, 2, 1)

im1 = plt.imshow(
    log_train,
    origin='lower',
    aspect='auto',
    vmin=min_scale,
    vmax=max_scale
)

plt.title("Training Error")
plt.xlabel("PCA Rank")
plt.ylabel("Delay Length")

num_delays, num_ranks = log_train.shape

# Grid lines at integer boundaries to distinguish better between consecutive cells
for x in range(num_ranks + 1):
    plt.axvline(x - 0.5, color="black", linewidth=0.3)
for y in range(num_delays + 1):
    plt.axhline(y - 0.5, color="black", linewidth=0.3)

plt.xticks(np.arange(num_ranks), ranks)
plt.yticks(np.arange(num_delays), delays)

cbar1 = plt.colorbar(im1)
cbar1.set_label("Log RMSE") # Colour bar Label

# Test error heatmap
plt.subplot(1, 2, 2)

im2 = plt.imshow(
    log_test,
    origin='lower',
    aspect='auto',
    vmin=min_scale,
    vmax=max_scale
)

```

```

)

plt.title("Testing Error")
plt.xlabel("PCA Rank")
plt.ylabel("Delay Length")

for x in range(num_ranks + 1):
    plt.axvline(x - 0.5, color="black", linewidth=0.3)
for y in range(num_delays + 1):
    plt.axhline(y - 0.5, color="black", linewidth=0.3)

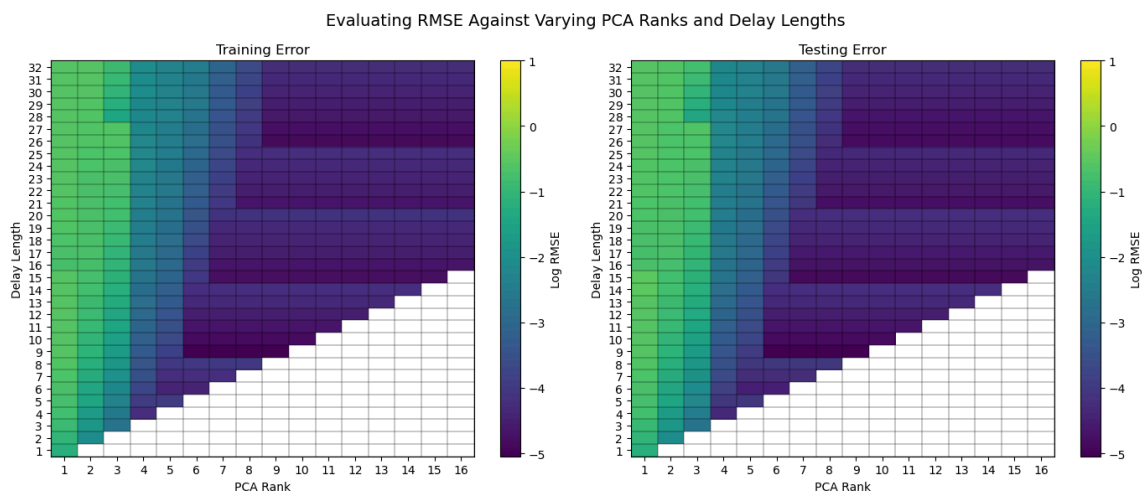
plt.xticks(np.arange(num_ranks), ranks)
plt.yticks(np.arange(num_delays), delays)

cbar2 = plt.colorbar(im2)
cbar2.set_label("Log RMSE")

plt.tight_layout()
plt.show()

```

Result :



The image shows how the logarithm of the RMSE varies with the PCA rank and delay length. Generally, both training and testing display similar trends in RMSE as it varies with the two independent variables, showing that there are minimal signs of overfitting. For a fixed delay length of 32, as the PCA rank increases, the error decreases and using PCA rank of 4, as the delay length increases, the error increases. Both training and error plots do show that the lowest error seemed to be at delay length 9 and between PCA ranks 6 and 9, inclusive. For PCA ranks above 8, there is a periodic variation in the error at delay lengths: 9, 15, 21 and 26 due to the oscillatory nature of the double beam system. At these delay lengths, the delay embedding captures the dynamics more effectively, resulting in lower error, however between those delay lengths, the embedding becomes less accurate and the error increases.

Problem 2

Question 1

```
In [ ]: def split_data(data_filepath, test_fraction=0.2, seed=18, output_filename=None):

    data = np.load(data_filepath)
    trajectories = [data[key] for key in data.files]

    np.random.seed(seed)
    N = len(trajectories)
    n_test = int(test_fraction * N)
    idx = np.random.permutation(N)

    test_idx = idx[:n_test]
    train_idx = idx[n_test:]

    train_trajectories = [trajectories[i] for i in train_idx]
    test_trajectories = [trajectories[i] for i in test_idx]

    if output_filename is not None:
        np.savez(output_filename,
                 train_indices=train_idx,
                 test_indices=test_idx)

    return train_trajectories, test_trajectories
```

Trajectory split is also applied for this problem since it allows for fair training and testing on unseen data, preventing data leakage between the two datasets.

Question 2

The DMD model accuracy must be evaluated in the reduced coordinate system, Z . DMD follows an invariant foliation structure as shown in Figure 2 where the original coordinates, x , and the delay embedded coordinates are reduced by an encoder, U . In Z , the dynamics are approximated using a linear operator, so the error needs to be assessed in the reduced space by comparing the mapping of the reduced coordinates.

Question 3

To order the dynamic modes, residuals can be used as a suitable metric. Modes with smaller residuals are considered more important, whereas larger residuals imply that a mode is less important.

The residual DMD measures how well a DMD-computed eigenvalue-eigenvector pair approximates a true Koopman eigenvalue-eigenvector pair. This metric is quantified using the discrepancy between the two pairs given by the formula below:

$$\text{res}(\lambda, h) = \frac{\|Kh - \lambda h\|^2}{\|h\|^2}.$$

where λ represents the DMD computed eigenvalue, h is the Koopman eigenfunction and K is the Koopman operator estimated from the data.

Question 4

```

In [ ]: # Error arrays
train_err = np.full((len(all_delays), len(all_ranks)), np.nan)
test_err = np.full((len(all_delays), len(all_ranks)), np.nan)

# Loop through all delay lengths
for di, delay in enumerate(all_delays):

    # 1) Delay embedded data in physical space
    X_train_delay, Y_train_delay = delay_embedding(train_trajectories, delay)
    X_test_delay, Y_test_delay = delay_embedding(test_trajectories, delay)

    # Filtering invalid columns
    if X_train_delay.shape[1] == 0 or X_test_delay.shape[1] == 0:
        continue

    # 2) Reduce using SVD once per run
    U, s, Vh = la.svd(X_train_delay, full_matrices=False)
    svd_dim = min(max_rank, len(s))

    U_r = U[:, :svd_dim]
    Z_train = U_r.T @ X_train_delay # Reduced coordinates
    Z_train_next = U_r.T @ Y_train_delay

    Z_test = U_r.T @ X_test_delay
    Z_test_next = U_r.T @ Y_test_delay

    # 3) Linear operator K in reduced space Z
    # K_full solves Z_train_next ≈ K_full Z_train
    Gz = Z_train @ Z_train.T
    K_full = Z_train_next @ Z_train.T @ la.pinv(Gz, rcond=1e-9)

    # 4) residual DMD to order dynamic modes for this delay

    # eigenpairs of K_full^T
    values, vectors = la.eig(K_full.T)
    right_vectors = la.inv(vectors)

    # Gram matrices in reduced coordinates Z
    G = Z_train @ Z_train.T
    H = Z_train_next @ Z_train.T
    L = Z_train_next @ Z_train_next.T

    # Residual for each eigenpair (lambda_k, v_k)
    res_num = []
    res_den = []
    for k in range(vectors.shape[0]):
        vk = vectors[:, k]
        lam = values[k]

        num = vk.conj().T @ (L - lam * H - np.conj(lam) * H.T + (np.abs(lam) ** 2))
        den = vk.conj().T @ G @ vk

        res_num.append(num)

```



```

        res_den.append(den)

res = np.real(np.array(res_num) / np.array(res_den)).flatten()

# smallest residual = most important mode
idx_sorted = np.argsort(res)

# Cannot keep more nodes than eigenvalues or dim(svd)
max_modes_here = min(max_rank, len(values), svd_dim)

# Loop through all ranks
for ri, r in enumerate(all_ranks):
    if r > max_modes_here:
        continue

    # Projection onto subspace spanned by r most accurate modes
    sel = idx_sorted[:r]

    # Projection operator
    DMD_proj = right_vectors[sel, :].T @ vectors[:, sel].T
    K_r = np.real(K_full @ DMD_proj)

    # Prediction in reduced space
    Z_train_next_pred = K_r @ Z_train
    Z_test_next_pred = K_r @ Z_test

    # Reconstruct to delay space
    Ytr_pred = U_r @ Z_train_next_pred
    Yte_pred = U_r @ Z_test_next_pred

    # RMSE
    dis_tr = Ytr_pred[-1, :] - Y_train_delay[-1, :]
    dis_te = Yte_pred[-1, :] - Y_test_delay[-1, :]

    train_err[di, ri] = np.sqrt(np.mean(dis_tr**2))
    test_err[di, ri] = np.sqrt(np.mean(dis_te**2))

# 7) best delay and rank combination by minimal test error
best_flat = np.nanargmin(test_err)
best_di, best_ri = np.unravel_index(best_flat, test_err.shape)

best_delay = all_delays[best_di]
best_rank = all_ranks[best_ri]

print(f"The best DMD model uses a delay length of {best_delay} and a DMD rank of {b

```

Result: The best model uses a delay length of 9 and a PCA rank of 6 which gives the lowest RMSE of 6.16e-06

Question 5

```

In [ ]: all_delays = range(1, max_delay + 1)
        all_ranks = range(1, max_rank + 1)

```

```

energy_error_train = np.full((len(all_delays), len(all_ranks)), np.nan)
energy_error_test  = np.full((len(all_delays), len(all_ranks)), np.nan)

for di, delay in enumerate(all_delays):

    X_train_delay, Y_train_delay = delay_embedding(train_trajectories, delay)
    X_test_delay, Y_test_delay = delay_embedding(test_trajectories, delay)

    if X_train_delay.shape[1] == 0 or X_test_delay.shape[1] == 0:
        continue

    U, s, Vh = la.svd(X_train_delay, full_matrices=False)
    svd_dim = min(max_rank, len(s))

    U_r = U[:, :svd_dim]
    Z_train = U_r.T @ X_train_delay
    Z_train_next = U_r.T @ Y_train_delay
    Z_test = U_r.T @ X_test_delay
    Z_test_next = U_r.T @ Y_test_delay

    Gz = Z_train @ Z_train.T
    K_full = Z_train_next @ Z_train.T @ la.pinv(Gz, rcond=1e-9)

    values, W = la.eig(K_full)

    # Use first reduced snapshot as reference for amplitudes
    z0 = Z_train[:, 0]

    # Getting modal amplitudes
    a, *_ = la.lstsq(W, z0, rcond=None)
    energy = np.abs(a)**2

    # Order modes from most to Least energetic
    idx_sorted = np.argsort(-energy) # descending order

    max_modes_here = min(max_rank, len(values), svd_dim)

    for ri, r in enumerate(all_ranks):
        if r > max_modes_here:
            continue

        # Select r most energetic modes
        sel = idx_sorted[:r]

        W_r = W[:, sel]
        Lambda_r = np.diag(values[sel])
        W_r_pinv = la.pinv(W_r)

        # Rank r operator from selected eigenpairs
        K_r = np.real(W_r @ Lambda_r @ W_r_pinv)

        Z_train_next_pred = K_r @ Z_train
        Z_test_next_pred = K_r @ Z_test

        Ytr_pred = U_r @ Z_train_next_pred

```

```

Yte_pred = U_r @ Z_test_next_pred

dis_tr = Ytr_pred[-1, :] - Y_train_delay[-1, :]
dis_te = Yte_pred[-1, :] - Y_test_delay[-1, :]

energy_error_train[di, ri] = np.sqrt(np.mean(dis_tr**2))
energy_error_test[di, ri] = np.sqrt(np.mean(dis_te**2))

best_flat = np.nanargmin(energy_error_test)
best_di, best_ri = np.unravel_index(best_flat, energy_error_test.shape)

best_delay_energy = all_delays[best_di]
best_rank_energy = all_ranks[best_ri]
best_rmse = energy_error_test[best_di, best_ri]

print("Best delay length (energy-ordered DMD):", best_delay_energy)
print("Best DMD rank (energy-ordered):", best_rank_energy)
print(f"Best test RMSE (energy-ordered): {best_rmse:.2e}")

```

Results: Best delay length (energy-ordered DMD): 9 Best DMD rank (energy-ordered): 9 Best test RMSE (energy-ordered): 8.995131676471643e-06

The two methods of ordering used are residual and energy ordering. When applying these ordering methods, the outputted optimal rank is different for both. For the residual ordering, the optimal combination was a delay length of 9 and a DMD rank of 6 with an RMSE of 6.16×10^{-6} , while the energy ordering has an optimal delay length of 9 and a DMD rank of 9, attaining an RMSE of 9.00×10^{-6} which shows that even though energy ordering captures a lot of the variance, it still remains slightly worse than the residual ordering method.

Problem 3

Question 1

```

In [ ]: # Fixed parameters for finding best model
ORDERS = [1, 2, 3]
MAX_RANK = 120
DELAY = 12

# Delay embedded by length 12
X_train, Y_train = delay_embedding(train_trajectories, DELAY)
X_test, Y_test = delay_embedding(test_trajectories, DELAY)

# Monomials up to order 3
def build_monomial_library(X, order):
    d, N = X.shape

    # Linear terms
    feats = [X]

    # Quadratic terms
    if order >= 2:

```

```

        quad = [X[i,:] * X[j,:] for i, j in combinations_with_replacement(range(d),
        feats.append(np.array(quad))

    # Cubic terms
    if order >= 3:
        cub = [X[i,:]*X[j,:]*X[k,:]
                for i,j,k in combinations_with_replacement(range(d), 3)]
        feats.append(np.array(cub))

    return np.vstack(feats)

# Train and test errors for all ranks for one order
def dmd_feature_space_errors(X_train, Y_train, X_test, Y_test, order, max_rank=120,

    # Builds feature space data
    Xphi_train = build_monomial_library(X_train, order)
    Yphi_train = build_monomial_library(Y_train, order)
    Xphi_test = build_monomial_library(X_test, order)
    Yphi_test = build_monomial_library(Y_test, order)

    U, S, Vt = la.svd(Xphi_train, full_matrices=False)
    V = Vt.T
    rmax = min(max_rank, len(S))

    # Store rank, training error and testing error
    ranks = []
    tr_err = []
    te_err = []

    idx_phys = delay - 1 # Index of last physical coordinate in delay-embedded spa

    for r in range(1, rmax + 1):

        Ur = U[:, :r]
        Sr = S[:r]
        Vr = V[:, :r]

        # Reduced Koopman operator in feature space
        K = Ur.T @ Yphi_train @ Vr @ np.diag(1/Sr)

        Xtr_r = Ur.T @ Xphi_train
        Xte_r = Ur.T @ Xphi_test

        Ytr_pred_r = K @ Xtr_r
        Yte_pred_r = K @ Xte_r

        Ytr_pred = Ur @ Ytr_pred_r
        Yte_pred = Ur @ Yte_pred_r

        # RMSE
        dis_train = Ytr_pred[idx_phys] - Yphi_train[idx_phys]
        dis_test = Yte_pred[idx_phys] - Yphi_test[idx_phys]

        error_train = np.sqrt(np.sum(dis_train**2) / dis_train.size)
        error_test = np.sqrt(np.sum(dis_test**2) / dis_test.size)

```

```

        ranks.append(r)
        tr_err.append(error_train)
        te_err.append(error_test)

    return np.array(ranks), np.array(tr_err), np.array(te_err)

all_ranks = None
train_errors = []
test_errors = []

# Used to track best model globally
best_error = np.inf
best_order = None
best_rank = None

for order in ORDERS:

    ranks, train, test = dmd_feature_space_errors(
        X_train, Y_train, X_test, Y_test,
        order=order,
        max_rank=MAX_RANK,
        delay=DELAY
    )

    # Align Length across orders
    if all_ranks is None:
        all_ranks = ranks
    else:
        L = min(len(all_ranks), len(ranks))
        all_ranks = all_ranks[:L]
        train = train[:L]
        test = test[:L]

    train_errors.append(train)
    test_errors.append(test)

    # Best rank for this order
    idx = np.argmin(test)

    # Update global best error by indices
    if test[idx] < best_error:
        best_error = test[idx]
        best_order = order
        best_rank = ranks[idx]

train_errors = np.array(train_errors)
test_errors = np.array(test_errors)

print(f" Best model order = {best_order}")
print(f" Best DMD rank   = {best_rank}")
print(f" Best test RMSE   = {best_error:.2e}")

```

Results:

Time: 1 min 59.7 seconds

The best model is a linear model with a DMD rank of 12.

Question 2

a) Since the library contains high order monomials and constants, the physical co-ordinates must be extracted by determining which monomials have a total order of 1. These monomials correspond to the original coordinates, so extracting them gives the physical coordinates.

b) If there are no linear monomials in the polynomial basis, then it is impossible to directly extract any physical co-ordinates. Hence, a decoder can be used to map the nonlinear monomial coefficients into approximate physical coordinates, which are then extracted. The reconstruction is given by the formula below:

$$\mathbf{x} = \mathbf{L}\mathbf{z}$$

where $\mathbf{x}$ is the vector of the physical coordinates, $\mathbf{L}$ is the linear operator and $\mathbf{z}$ is the coefficients in the nonlinear function.

c) Given the training data, the optimal way of choosing $\mathbf{L}$ is by using the least-square regression. This is done by minimising the reconstruction error, which is evaluated by calculating the RMSE using the residuals. The decoder, which results in the lowest RMSE, will be used as the best approximation to extract the physical coordinates.

Question 3

```
In [ ]: # Build feature libraries for the best order
Xphi_train_best = build_monomial_library(X_train, order=best_order)
Yphi_train_best = build_monomial_library(Y_train, order=best_order)

U_best, S_best, Vt_best = la.svd(Xphi_train_best, full_matrices=False)
V_best = Vt_best.T

r = min(best_rank, len(S_best))
Ur_best = U_best[:, :r]
Sr_best = S_best[:r]
Vr_best = V_best[:, :r]

# Reduced Koopman operator
K = Ur_best.T @ Yphi_train_best @ Vr_best @ np.diag(1.0 / Sr_best)

# Select the first test trajectory
test_traj = np.asarray(test_trajectories[0]).flatten()
T_total = len(test_traj)

# First 240 steps for comparison
num_steps = 240

# Build the initial delay-embedded window (shape (DELAY,))
```

```

init_block = test_traj[:DELAY].reshape(DELAY, 1)

# Lift initial block into feature space
Phi = build_monomial_library(init_block, order=best_order)

# Reduced coordinates
a = Ur_best.T @ Phi

# Store predictions
y_pred = []

idx_phys = DELAY - 1

for k in range(num_steps):

    # Reconstruct feature vector
    Phi_k = Ur_best @ a

    # Extract the predicted physical value
    y_pred.append(Phi_k[idx_phys, 0])

    # Next state
    a = K @ a

y_pred = np.array(y_pred)

# True values
y_true = test_traj[DELAY:DELAY + num_steps]

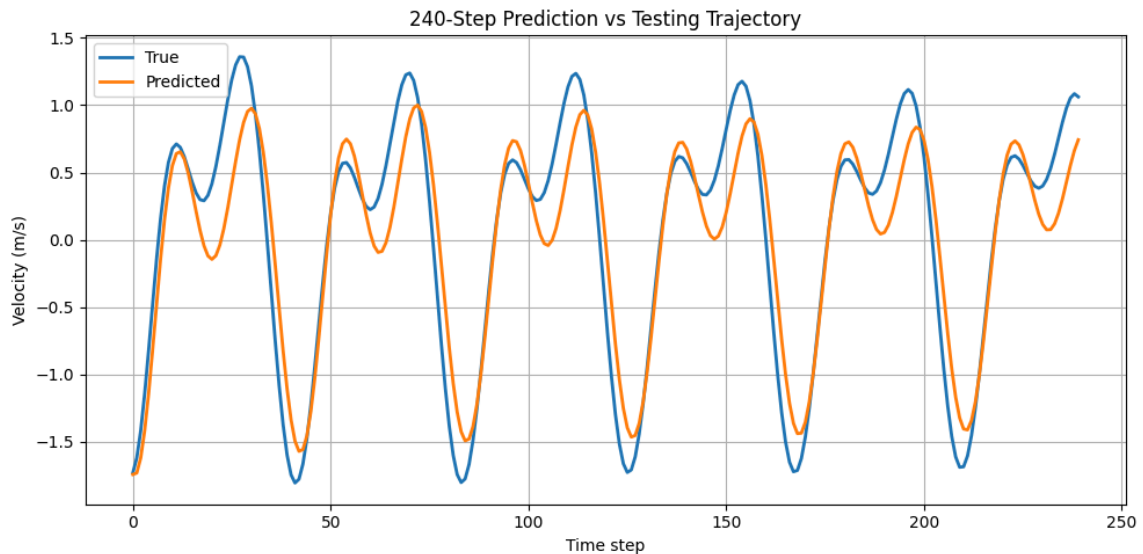
# Compute RMSE
rmse = np.sqrt(np.mean((y_pred - y_true)**2))

print("240-Step Simulation Results")
print(f"RMSE = {rmse:.2e}")

# Graph to compare the true vs predicted trajectory
plt.figure(figsize=(10, 5))
plt.plot(y_true, label="True", linewidth=2)
plt.plot(y_pred, label="Predicted", linewidth=2)
plt.xlabel("Time step")
plt.ylabel("Velocity (m/s)")
plt.title("240-Step Prediction vs Testing Trajectory")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Results:



The graph above shows that the predicted trajectory using the model with order 1 and DMD rank 12 and the true values of the velocity over the first 240 samples of the testing trajectory. The predicted model exhibits reasonable accuracy with an RMSE of approximately 0.35. The main sources of error comes from the peaks and troughs where the predicted trajectory fell short on the amplitudes of the true trajectory. Regardless, the DMD linear model is still able to capture a good portion of the system dynamics over a relatively long time period.

Problem 4

Question 1

```
In [ ]: # Creating Lasso regression solver using coordinate descent
def lasso(XX, YY, alpha, eps, maxiter=5000):
    GG = XX @ XX.T
    HH = YY @ XX.T

    # Soft threshold function enforcing the L1 penalty
    def softmax(x, alpha):
        if np.abs(x) > alpha:
            return x - alpha * np.sign(x)
        else:
            return 0.0 # Small coefficients set to zero

    WW = np.zeros_like(HH)
    WW_next = np.zeros_like(HH)

    for k in range(maxiter):
        for (p, q), z in np.ndenumerate(HH):
            b = HH[p, q] + WW_next[p, q] * GG[q, q] - np.dot(WW_next[p, :],
                GG[:, q])
            # Regression equation
            WW_next[p, q] = softmax(b, alpha) / GG[q, q]

        if la.norm(WW - WW_next) < eps: # Convergence check
            return WW_next
```



```

        np.copyto(WW, WW_next)

    return WW_next

# RMSE
def compute_rmse(YY_true, YY_pred):
    diff = (YY_pred - YY_true)**2
    per_coord_mse = np.mean(diff, axis=1)    # Average over samples
    return np.sqrt(np.mean(per_coord_mse))    # Average over coordinates

def best_alpha(data_filepath: str,
               delay: int = 12,
               test_fraction: float = 0.2,
               seed: int = 18):

    # Delay embedding
    XX_train, YY_train = delay_embedding(train_trajectories, delay)
    XX_test, YY_test = delay_embedding(test_trajectories, delay)

    alpha_list = np.logspace(-6, 3, 60)    # Range of alpha, the regularisation param

    # Storing results in arrays to select best alpha and for plotting
    err_train_list = []
    err_test_list = []
    terms_list = []
    W_list = []

    for alpha in alpha_list:
        WW_l = lasso(XX_train, YY_train, alpha, eps=2 ** (-24))

        W_list.append(WW_l)

        YY_train_pred = WW_l @ XX_train
        err_train_list.append(compute_rmse(YY_train, YY_train_pred))

        YY_test_pred = WW_l @ XX_test
        err_test_list.append(compute_rmse(YY_test, YY_test_pred))

        terms_list.append(np.count_nonzero(WW_l))

    # Training and testing errors
    err_train_list = np.array(err_train_list)
    err_test_list = np.array(err_test_list)
    terms_list = np.array(terms_list)

    # For picking best alpha based on minimum testing error
    idx_best = np.argmin(err_test_list)

    alpha_best = alpha_list[idx_best]
    W_best = W_list[idx_best]
    n_nonzero_best = terms_list[idx_best]

```

```

print(
    f"The best alpha is {alpha_best:.3g} with testing RMSE: "
    f"{err_test_list[idx_best]:.2e} and {n_nonzero_best} nonzero parameters."
)

return {
    "alpha_list": alpha_list,
    "err_train_list": err_train_list,
    "err_test_list": err_test_list,
    "terms_list": terms_list,
    "alpha_best": alpha_best,
    "W_best": W_best,
    "n_nonzero_best": n_nonzero_best
}

# Graph for training and testing RMSE against alpha
def graph(results):
    alpha_list = results["alpha_list"]
    err_train_list = results["err_train_list"]
    err_test_list = results["err_test_list"]

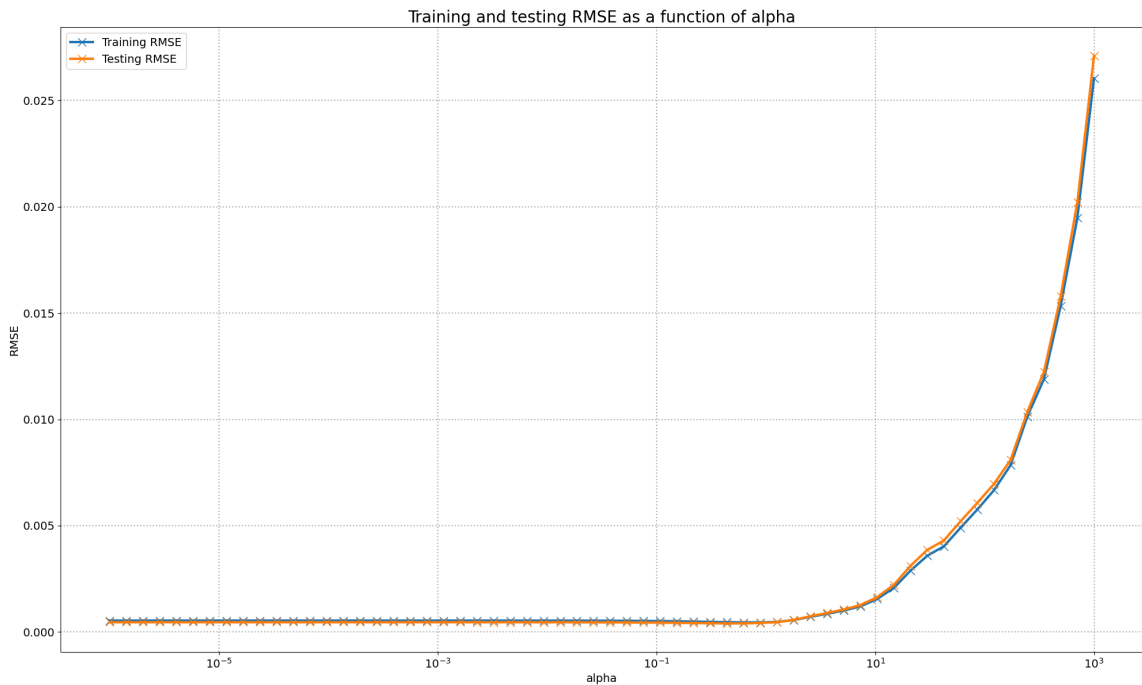
    plt.figure(figsize=(20, 12))
    plt.semilogx(alpha_list, err_train_list, marker='x', markersize=10,
        linewidth=3, label='Training RMSE')
    plt.semilogx(alpha_list, err_test_list, marker='x', markersize=10,
        linewidth=3, label='Testing RMSE')
    plt.xlabel('alpha', fontsize=14)
    plt.ylabel('RMSE', fontsize=14)
    plt.title('Training and testing RMSE as a function of alpha', fontsize=20)
    plt.xticks(fontsize=14)
    plt.yticks(fontsize=14)
    plt.grid(True, which='both', linestyle=':', linewidth=1.5)
    plt.legend(fontsize=14)
    plt.tight_layout()
    plt.show()

results = best_alpha("double_beam_data.npz", delay=12, test_fraction=0.2, seed=18)
graph(results)

```

Results:

Time: 2 mins 33 seconds



The best alpha is 0.441 with testing RMSE: 3.99e-04 and 97 nonzero parameters.

The training and testing error curves display similar trends and sit very close to each other across the whole range of α , which indicates that there is minimal signs of overfitting occurring. At extremely low values of alpha, the RMSE decreases gradually until the optimal value of 0.441 where it reaches the minimum testing error of 3.99×10^{-4} . This is the point at which the fitting of the data and the sparsity of the data is balanced most effectively. For values of alpha larger than 0.441, the RMSE increases much more rapidly. This is due to the fact the soft threshold function which could lead to a lot more coefficients being removed which leads to the model being overly sparse. This increases the training and testing RMSE, as a result of underfitting.

Problem 5

Question 1

```
In [ ]: def fmt(x):          # Formatting of values to 3 significant figures
        if x == 0:
            return "0.00"
        p = int(np.floor(np.log10(abs(x))))
        dec = max(0, 2 - p)
        return f"{x:.{dec}f}"

        data_dict = np.load('double_beam_data.npz')
        data_all = [it[1] for it in data_dict.items()]

        XX, YY = delay_embedding(data_all, DELAY)

        # Fitting dense linear model
        W = (YY @ XX.T) @ la.pinv(XX @ XX.T, rcond=1e-6)
```

```

eigvals, _ = la.eig(W) # Computing eigenvalues

# Filter out pure eigenvalues since they're not oscillatory
osc = eigvals[np.abs(np.imag(eigvals)) > 1e-6]

# Extract unique positive angles (group conjugate pairs)
angles = np.abs(np.angle(osc))
unique_angles = np.unique(np.round(angles, 6))

dt = 1/5120 # time step between measurements (s)

def compute_modes(W, dt):
    eigvals, _ = la.eig(W)

    osc = eigvals[np.abs(np.imag(eigvals)) > 1e-9]
    angles = np.abs(np.angle(osc))

    unique = np.unique(np.round(angles, 6))

    omega = []
    zeta = []

    for ang in unique:
        lam = osc[np.isclose(np.abs(np.angle(osc)), ang)][0]
        omega.append(ang / dt)
        zeta.append(-np.log(np.abs(lam)) / ang)

    omega = np.array(omega)
    zeta = np.array(zeta)

    idx = np.argsort(omega)
    return omega[idx], zeta[idx]

# Fit dense linear model
W = (YY @ XX.T) @ la.pinv(XX @ XX.T, rcond=1e-6)

# Extract dominant modes
omega_list, zeta_list = compute_modes(W, dt)

for i in range(len(omega_list)):
    print(f"Mode {i+1} has a natural frequency of {fmt(omega_list[i])} rad/s and a

```

Results :

Mode 1 has a natural frequency of 768 rad/s and a damping ratio of 0.00239.

Mode 2 has a natural frequency of 1531 rad/s and a damping ratio of 0.00266.

Question 2

```

In [ ]: # Full DMD Operator
W_full = (YY @ XX.T) @ la.pinv(XX @ XX.T, rcond=1e-6)

# Compute all modes for the full model

```

```

omega_all, zeta_all = compute_modes(W_full, dt)

dom_indices = []
dom_omegas = []
freq_tol = 1e-3 # tolerance on frequency

for idx in sorted_idx:
    w = omega_all[idx]
    if all(abs(w - w0) > freq_tol for w0 in dom_omegas):
        dom_indices.append(idx)
        dom_omegas.append(w)
    # Select two distinct least damped modes
sorted_idx = np.argsort(zeta_all) # Using smaller zeta
if len(dom_indices) == 2:
    break

# Reference values from operator
omega_ref = np.array(dom_omegas)
zeta_ref = zeta_all[dom_indices]

print("Reference dominant modes (from full model):")
for j in range(2):
    print(f"\nMode {j+1}:")
    print(f"omega_ref = {omega_ref[j]:.3f} rad/s")
    print(f"zeta_ref = {zeta_ref[j]:.5e}")

# Bagging parameters
B = 200 # 200 bootstrap models
frac = 0.8
N = XX.shape[1]

omega_samples = np.zeros((B, 2))
zeta_samples = np.zeros((B, 2))

for b in range(B):

    # Sample 80% of columns at random
    idx = sorted(random.sample(range(N), int(frac * N)))

    XX_sel = XX[:, idx]
    YY_sel = YY[:, idx]

    Wb = (YY_sel @ XX_sel.T) @ la.pinv(XX_sel @ XX_sel.T, rcond=1e-6)

    # Computes for all modes for this bootstrap operator
    omega_b, zeta_b = compute_modes(Wb, dt)

    # For each reference mode, find closest mode in this bootstrap by omega
    for j in range(2):
        k = np.argmin(np.abs(omega_b - omega_ref[j]))
        omega_samples[b, j] = omega_b[k]
        zeta_samples[b, j] = zeta_b[k]

```

```

# Fitting Gaussian distribution
results = {}

for mode in [0, 1]:

    # data shape (B, 2): [omega, zeta]
    data = np.vstack([omega_samples[:, mode], zeta_samples[:, mode]]).T

    mean_vec = np.mean(data, axis=0)
    cov_mat = np.cov(data.T) # Covariance matrix

    results[f"Mode {mode+1}"] = {"mean": mean_vec, "cov": cov_mat}

for m in ["Mode 1", "Mode 2"]:
    mean = results[m]["mean"]
    cov = results[m]["cov"]

    print(m)
    print(f"Mean  $\omega$  = {fmt(mean[0])} rad/s")
    print(f"Mean  $\zeta$  = {fmt(mean[1])}")
    print("Covariance matrix:")
    print(cov)
    print()

```

Result :

Using the Gaussian distribution fitting, Mode 1 has an estimated mean natural frequency of 768 rad/s and a mean damping ratio of 0.000971

Covariance matrix:

```

[ [ 0.00167 , 0.00000000613 ]
  [ 0.00000000613 , 0.00000000206 ] ]

```

Using the Gaussian distribution fitting, Mode 2 has an estimated mean natural frequency of 1526 rad/s and a mean damping ratio of 0.00222

Covariance matrix:

```

[ [ 0.0139 , 0.0000000936 ]
  [ 0.0000000936 , 0.00000000443 ] ]

```